

Rayan Morais

Engenheiro de Software Full-Stack

rayan_cm2021@icloud.com · linkedin.com/in/ravancmorais · Juiz de Fora, MG — Brasil · Inglês C1
github.com/ravancmorais · cupommaniac.com.br · ravancmorais.com.br

PERFIL

Engenheiro de Software Full-Stack com projetos reais em produção e foco em decisões de arquitetura justificadas. Forte domínio de TypeScript, Node.js, React e Next.js. Experiência prática com arquiteturas orientadas a eventos (RabbitMQ), microsserviços com DDD, filas de jobs (BullMQ/Redis), autenticação OAuth2/OIDC, gateways de pagamento (Stripe, PayPal, MercadoPago) e deploy em nuvem (AWS Lambda, Vercel, Railway). Valorizo código que sobrevive à produção: error handling explícito, refatorações orientadas a dados e documentação que serve ao próximo desenvolvedor. Trajetória anterior em saúde e direito consolidou tolerância zero ao erro e atenção a edge cases. Inglês C1.

EXPERIÊNCIA

Estagiário de Desenvolvimento Full-Stack

Fev 2026 – Presente

Vasc-Review — plataforma de educação médica continuada para residentes e cirurgiões vasculares

vascreview.com.br

- ▶ **Desenvolvimento front-end web e mobile (React Native):** nova versão da plataforma com videoaulas e banco de questões para residência e qualificação em cirurgia vascular — app publicado na App Store e Play Store, nova versão em desenvolvimento ativo.
- ▶ **Atuação em ambiente de produção real:** entrega de features, correção de bugs e manutenção de código utilizado por médicos especialistas, sob processo formal de revisão de código.

Desenvolvedor Full-Stack Freelance

2024 – Presente

Projetos independentes & colaboração em equipe

- ▶ **Colaboração com Engenheiro de Software Sênior** (atualmente na AT&T) em projetos full-stack — pair programming, code review e entrega de features em ambiente de produção real.
- ▶ **Entrega de aplicações web completas** com foco em decisões justificadas: escolha de stack, modelagem de banco, deploy e configuração de DNS.
- ▶ **Diagnóstico e resolução de bugs em produção:** identificação de N+1, race conditions em upserts e vulnerabilidades de autenticação, além de integração com APIs de terceiros (afiliados, OAuth, pagamento, email) com tratamento explícito de rate limits e retries.

Analista de Suporte de TI

2011 – 2014

Câmara Municipal de Madre de Deus – MG

- ▶ Administração de infraestrutura de redes e hardware com alta disponibilidade em ambiente institucional.
- ▶ Gerenciamento do ciclo de vida completo de chamados técnicos e resolução de problemas de conectividade.

HABILIDADES TÉCNICAS

Linguagens TypeScript · JavaScript (ES6+) · Go (Golang – Intermediário) · SQL · Python (básico)

Frontend Next.js 15 · React 19 · React Native · Tailwind CSS · Framer Motion · Styled Components · Radix UI

Backend Fastify · NestJS (Controllers/Services) · Node.js · Express · Go/Gin · BullMQ · RabbitMQ · Redis (Queues/Workers) · Socket.io

Banco de dados PostgreSQL · MongoDB · MySQL · Supabase · Prisma ORM · Meilisearch

Auth & Pagamentos OAuth2/OIDC · NextAuth v5 · Keycloak · JWT · Stripe · PayPal · MercadoPago

Cloud & DevOps AWS Lambda · AWS SAM · AWS Secrets Manager · Docker · Vercel · Railway · GitHub Actions · Kong API Gateway

Arquitetura, Padrões & Qualidade SOLID · DDD (Domain-Driven Design) · Clean Architecture · Arquitetura Orientada a Eventos (EDA) · Microsserviços · Serverless · Controllers, Services & Workers (NestJS) · TDD (Vitest/Jest/Testing Library) · Clean Code · Design Tokens & Componentização Dinâmica

Serviços Cloudinary · Resend · Sentry · Anthropic Claude (API) · Pix (MercadoPago)

Design & Tooling Figma (High-Fidelity) · React Hook Form + Zod · ESLint · Prettier · Husky · Knip · Git · Postman · Linux

PROJETOS

CupomManiac — Plataforma de Cupons de Desconto

2026

cupommaniac.com.br · github.com/ravancmorais/cupomManiac

Plataforma fullstack de cupons para o mercado brasileiro (similar ao Méliuz), desenvolvida do zero até produção com domínio próprio.

- ▶ **Refatoração de N+1 no sync de afiliados:** cada ciclo trazia milhares de cupons de fontes como Lomadee e Admitad processados em upserts individuais, travando o banco sob carga. Reescrito como upsertMany com deduplicação em batch por (fonte, idExterno), indexando no Meilisearch em paralelo.
- ▶ **Pooler do Neon + singleton de PrismaClient:** o tier gratuito do Neon limita a 10 conexões diretas, e Server Components do Next.js instanciando múltiplos PrismaClient estourava o limite em pico. Migrado para o endpoint pooler com client singleton no backend.

- **Proxy routes para contornar cookies cross-origin:** backend (Railway) e frontend (Vercel) em domínios diferentes faziam SameSite=Lax bloquear cookies em mutations via fetch. Mutations sensíveis passam por rotas proxy no Next.js que validam a sessão server-side antes de repassar ao backend.
- **AlertMatcherService com O(1) roundtrips:** matching naive de alertas exigiria N queries por cupom novo. Reescrito como findMany → createMany (skipDuplicates) → updateMany, independente do volume de cupons no ciclo de sync.
- **5 fontes de afiliados via Adapter Pattern:** Lomadee, Admitad, Awin, Amazon Associates e Mercado Livre implementam a mesma interface de sync — adicionar um fornecedor não altera o service de upsert. Cron jobs automáticos rodam sync a cada 3h e limpeza de expirados diariamente às 2h.
- **Sistema de alertas com double opt-in e scoring algorítmico (0–100):** verificação por email → confirmação → matching em batch → alerta com unsubscribe por token; cupons ranqueados por desconto, presença de código e proximidade da expiração.

Stack: Next.js 15 (App Router, output standalone), TypeScript, Fastify, Prisma ORM, PostgreSQL (Neon), Redis (Upstash), BullMQ, Meilisearch, NextAuth v5, Resend, Sentry, Tailwind CSS, Vault, Vercel, Railway

Crash Game — Desafio Técnico Fullstack (Jungle Gaming)

2026

github.com/rayancmorais/fullstack-challengeRayancm

Crash game multiplayer em tempo real: multiplicador sobe de 1.00x e pode crashar — jogadores apostam e sacam antes do crash.

- **Game Service e Wallet Service comunicam exclusivamente via RabbitMQ** — sem HTTP direto entre serviços. Decisão: Wallet precisa ser a única fonte de verdade para saldo; HTTP síncrono criaria acoplamento e risco de inconsistência em falhas parciais.
- **DDD com 4 camadas:** a camada domain é TypeScript puro, zero NestJS, zero Prisma — regras de negócio testáveis sem mock de framework. 91 testes unitários + 11 cenários E2E cobrindo bet→cashout→saldo, bet→crash→perdido e validações de limite.
- **Valores monetários como BigInt centavos** (PostgreSQL BIGINT, TypeScript BigInt): R\$1,50 = 150n — elimina erros de ponto flutuante em cálculos financeiros. DTOs serializam como string para contornar limitação do JSON.stringify.
- **Provably fair com HMAC-SHA256:** hashSeedServidor exposto antes das apostas, seed revelado após o crash — jogador pode recalcular o ponto de crash independentemente e verificar que não houve manipulação.
- **Kong API Gateway (DB-less)** para rate limiting e roteamento REST; WebSocket conecta direto no Game Service (porta 4001) — Kong em DB-less tem limitações com upgrade HTTP→WS. CI via GitHub Actions com build, test, lint e build-check Docker antes de merge.

Stack: NestJS, TypeScript, Bun, Next.js 15, PostgreSQL, Prisma 7, RabbitMQ, Kong, Keycloak, Socket.io, Zustand, TanStack Query, Docker, GitHub Actions

Lux Lab Brasil — Plataforma de Dropshipping (BR + EU)

2025

luxlabbr.vercel.app · repositório privado, disponível sob solicitação

Plataforma fullstack de dropshipping multi-mercado (Brasil e União Europeia), com fornecedores internacionais, múltiplos gateways de pagamento, i18n e assistente de IA.

- **Multi-fornecedor com roteamento geográfico:** interface única Supplier (Name(), CreateOrder(), WebhookURL()) implementada por fornecedores BR e EU (BigBuy, Griffati, Printful, entre outros); PreferredForCountry(country) retorna a lista ordenada por país de entrega — adicionar um fornecedor não altera o fluxo de pedidos. Sincronização de catálogo paralela via goroutines concorrentes.
- **Pipeline de precificação por região:** normalização de preço do fornecedor para a moeda local (BRL ou EUR), cálculo de margem bruta e alíquota de imposto por país de destino (IVA/VAT) — três etapas isoladas e testáveis independentemente.
- **OAuth2/OIDC multi-provider com PKCE:** Google, Facebook e Apple — state assinado via OAUTH_STATE_SECRET. Middleware duplo (jose.jwtVerify no Next.js edge, golang-jwt no Go) com roles user/admin/worker mapeados em ambas as camadas.
- **Chat com IA em streaming (SSE) sob restrição de Lambda:** API Gateway corta conexões em 29s, inviabilizando streams longos. Sessões de chat com Claude (Anthropic) limitadas a ≤28s com timeout explícito no handler, documentado para futura migração a WebSocket API.
- **Segurança de segredos em produção:** após exposição acidental de credenciais em .env, implementado gitleaks no CI para bloquear commits com segredos, com rotação imediata e migração de variáveis sensíveis para AWS Secrets Manager.

Stack: Go, Gin, MongoDB Atlas, Next.js 15, TypeScript, Tailwind CSS v4, Radix UI, React Hook Form + Zod, AWS Lambda, AWS SAM, AWS Secrets Manager, Stripe, PayPal, MercadoPago, Cloudinary, Resend, next-intl, Claude (Anthropic), GitHub Actions

Portfólio Pessoal

2025

rayancmorais.com.br · github.com/rayancmorais/myportfoliooficial

- **Design system por tokens CSS:** todo o visual controlado por CSS custom properties (--cy, --fg-1, --bg-1...); troca de tema dark/light via atributo data-theme no <html>, sem re-render de componentes.
- **Contextos próprios sem biblioteca de state:** ThemeContext e LanguageContext implementados do zero com createContext + useCallback, estado de tema persistido em localStorage — zero dependência de Redux ou Zustand.
- **Scroll suave sem recriar o loop a cada render:** o hook useLenis original instanciava o Lenis dentro do componente, então cada re-render recriava o loop de scroll e causava jank perceptível. Resolvido com singleton a nível de módulo — a instância vive fora do componente e sobrevive a re-renders.
- **Parallax fluido sem custo de bundle:** uma lib de parallax pronta adicionaria peso só para um efeito simples nas screenshots do case studies. Implementado à mão com getBoundingClientRect + requestAnimationFrame, com useReducedMotion() checado em toda animação para não forçar movimento em quem desativa isso no SO.

Stack: React 19, TypeScript, Vite, Styled Components, Framer Motion, Lenis, i18next, Vitest, Testing Library, ESLint, Prettier, Husky, Knip, Vercel

FORMAÇÃO

Bacharelado em Engenharia de Software

Em andamento · 2025

UNINTER

Bacharelado em Enfermagem

Concluído

UNIVERSO – Juiz de Fora